

dKargo

L3 Deployment/Operation Security Assessment Project

Final Report

January 2026

Theori

Theori, Inc. ("We") is acting solely for the client and is not responsible to any other party.

Deliverables are valid for and should be used solely in connection with the purpose for which they were prepared as set out in our engagement agreement. You should not refer to or use our name or advice for any other purpose.

The information (where appropriate) has not been verified. No representation or warranty is given as to accuracy, completeness or correctness of information in the Deliverables, any document, or any other information made available. Deliverables are for the internal use of the client and may not be used or relied upon by any person or entity other than the client. Deliverables are confidential and are not to be provided, without our authorization (preferably written), to entities or representatives of entities (including employees) that are not the client, including affiliates or representatives of affiliates of the client

©2026. For information, contact Theori, Inc.

Executive Summary

dKargo provides services by operating an Arbitrum Orbit-based L3 chain for the blockchain-based processing of logistics and supply chain data.

Theori received the source code in scope for the assessment from dKargo and reviewed the source code for a total of 3 weeks from December 22, 2025, to January 9, 2026.

In this review, to identify practical threats that may occur during the deployment and operation phases, we focused on checking the access control system, upgrade authority concentration, DAC and Sequencer operation parameters, and the accuracy of bridge address mapping. Additionally, we reviewed the call paths of key contracts based on various scenarios derived through threat modeling.

The purpose of this assessment was to identify potential security risks in the dKargo L3 chain and establish improvement measures. The results of the assessment can be summarized as follows.

	Description
Security Level Assessment Results - Excellent Items	• The standard bridge token pairing identifiable on-chain is consistently configured. • The basic address mapping of the L1 <> L2 and L2 <> L3 bridge paths is correctly set as intended by the design. • The basic mapping configuration using a custom gas token is normal.
Security Level Assessment Results - Improvement Needed Items	• Upgrade and key authorities are concentrated in a single entity, presenting a single point of failure (SPoF) risk. • The Data Availability Committee (DAC) operational configuration and committee distribution are not sufficiently secured, increasing the risk of reduced availability and censorship.

	• The Sequencer/BatchPoster is not distributed, presenting risks of delay/censorship and MEV. • Criteria for changing operational settings and the monitoring system are not sufficiently documented. • The appropriate distribution of Validators and Challengers cannot be confirmed on-chain, and actual participation metrics are not verified on-chain, requiring verification of the operational configuration. • A prior verification and approval process for deployment/operation settings (bridge path, gas token requirements, deployment parameters) is required.
Potential Risks	• Asset loss may occur due to malicious upgrades and state finalization in the event of upgrade authority concentration or abuse of Fast Confirmer authority. • Chain halts, censorship, and withdrawal delays may occur in the event of DAC/Sequencer failure or suspension of operation. • Deposit/withdrawal failures or escrow discrepancies may occur in the event of bridge path and gas token configuration errors. • L2/L3 deployment failures and service delays may occur in the event of misconfiguration of deployment parameters. • The risk of incorrect state finalization increases if verification power weakens due to low Validator participation.
Security Enhancement Proposals	• Distribute key authorities (upgrade, state finalization, administrator) through Multisig and Timelock, and clarify the change approval process. • Expand the number of DAC members and the threshold, establish a data availability monitoring system, and configure the Fallback mechanism to operate normally.

	• Distribute the Sequencer/BatchPoster and clearly review the criteria for changing operational setting values. • Operate a checklist and approval process during deployment/operation changes to verify bridge paths and gas token requirements in advance. • Inspect incident response solutions and monitoring metrics, and manage the distribution and participation metrics of Validators and Challengers.
--	---

Table of Contents

1. About This Document	7
2. Overview	8
2.1. Project Introduction	8
2.2. Project Purpose and Goals	8
2.3. Project Scope	8
3. Methodology	10
3.1. Project Execution Method	10
3.2. Threat Modeling	11
3.2.1. General Threat Modeling	11
3.2.2. Real-world Operation (On-chain) Based Threat Modeling	16
4. Findings	20
4.1. Findings Summary	20
4.2. Findings - On-chain Based Scenario Review	24
4.2.1. DKARGO-001: Admin/Upgrade/FastConfirmer Authority Concentration and Abuse	24
4.2.2. DKARGO-002: Abuse of Deployment mapping override	27
4.2.3. DKARGO-003: DAS Keyset and DAC Operation Failure	28
4.2.4. DKARGO-004: Sequencer/BatchPoster Centralization and MEV Extraction	31
4.2.5. DKARGO-005: Sequencer Inbox Parameter (Delay Buffer) Setting Insufficiency	34
4.2.6. DKARGO-006: Validator Allowlist and Challenger Distribution Insufficiency	37
4.2.7. DKARGO-007: Staking Parameter Setting Discrepancy (stakeToken/baseStake)	39
4.3. Findings - Unverified On-chain Items and Assumption Based Scenario Review	41
4.3.1. DKARGO-008: Custom Gas Token Requirements, Pricing, and Template Approval Vulnerable Configuration	41
4.3.2. DKARGO-009: Token Standard, Metadata, and Decimal Compatibility Vulnerable Configuration	44

4.3.3. DKARGO-010: Gateway/Router Path and Fee Operation Threats	46
4.3.4. DKARGO-011: Rollup/Template Initialization Consistency Threats	48
4.4. Findings - Contract Scenario Based Review	50
4.4.1. DKARGO-012: Insufficient Atomicity Assurance in Deployment and Initialization Process	50
4.4.2. DKARGO-013: Sequencer Delay/Censorship and Force Inclusion	51
4.4.3. DKARGO-014: Retryable fee Parameter Calculation Error	52
4.4.4. DKARGO-015: AnyTrust/DAC Keyset Change Failure	53
4.4.5. DKARGO-016: Abuse of Fast Confirm Key	54
4.4.6. DKARGO-017: Routing Address Misconfiguration (Inbox, Outbox, Sequencer)	55
4.4.7. DKARGO-018: L2 Token Metadata Parsing Failure	56
5. Discussion	57
5.1. Operational Maturity Risk Management	57
5.2. Gelato RaaS Operational Risk Management	62
5.3. Operational Risk Management Based on Similar Project Incident Cases	64
6. Conclusion	66

1. About This Document

This report contains the details of the Deployment/Operation Security Assessment Project performed by Theori for a total of 3 weeks from December 22, 2025, to January 9, 2026, targeting the source code (dKargo/token-bridge-contracts, dKargo/nitro-contracts v3.1.0, totaling 10 contracts) received from dKargo.

The contents covered in each chapter of the report are as follows.

Chapter	Content
2. Overview	This part summarizes the nature of the project and the scope of execution.
3. Methodology	This part covers the feasibility of threat scenarios created through a combination of attack vectors and threats from an attacker's perspective, based on an analysis of the major services provided by dKargo.
4. Findings	This is the core part of the report describing the details of the dKargo security threats identified while performing the project. Each threat finding is broadly classified into three categories. More detailed information can be found in the description section for each threat: On-chain Based Scenarios, On-chain Unverified/Assumption Based Scenarios, and Contract Based Scenarios.
5. Discussion	This part covers risk management for third parties, on-chain, and similar project incident cases among the dKargo security threats identified while performing the project.
6. Conclusion	This contains an overall summary of the project results and Theori's opinion on the security of dKargo.

2. Overview

2.1. Project Introduction

Project Target	dKargo
Project Nature	Design Security Review to derive potential security threats through static/dynamic analysis
Project Period	December 22, 2025 - January 9, 2026 (3 weeks)

2.2. Project Purpose and Goals

To improve the security of the dKargo service through preemptive cyber threat response, the project was performed by applying a security assessment methodology specialized for major services and systems.

The detailed goals are as follows:

- Identify potential security threats within target assets for safe deployment/operation.
- Identify major threat possibilities during Gelato RaaS deployment/operation.
- Identify threat possibilities through investigation of similar project incident cases.
- Propose best practices for countermeasures against findings.

2.3. Project Scope

Theori performed the project targeting dKargo's service contracts.

Category	Classification	Target
dKargo Smart Contract	Contract	dKargo/token-bridge-contracts: 5cfcc2c7ba6b7ed33e9cbbd5659ca214979c7854 • StandardArbERC20.sol

		dKargo/nitro-contracts/tree/v3.1.0: 9d0e90ef588f94a9d2ffa4dc22713d91a76f57d4 • ERC20Inbox.sol • ERC20Bridge.sol • ERC20Outbox.sol • SequencerInbox.sol • EdgeChallengeManager.sol • RollupProxy.sol • RollupCore.sol • RollupAdminLogic.sol • RollupUserLogic.sol
--	--	--

※ The project scope targets only the source code (contracts) of dKargo. On-chain contracts were referenced for the purpose of checking the current deployment/operation status.

3. Methodology

3.1. Project Execution Method

The project was performed according to the process below.

#	Description
1	Understand the service through review of the project target white paper and development documents.
2	Understand the service by directly using the project target services.
3	Identify on-chain deployed contract addresses provided by the service.
4	Understand business logic through static/dynamic analysis of the provided source code (contracts).
5	Select points with potential threat occurrence based on the identified contents.
6	Derive threat scenarios through threat modeling.
7	Check actual threat occurrence based on derived threat scenarios (Threat Check).

3.2. Threat Modeling

3.2.1. General Threat Modeling

Classification	Content	ID Code
Operation Threat Based	Is there a risk of incorrect state root finalization and fraudulent withdrawal occurring because verification is not performed due to collusion between the proposer and Challengers, given the limited number of external Challengers?	DKARGO-006
	Is there a risk that withdrawals occur without a challenge period because malicious state roots are quickly finalized due to the concentration of Fast Confirmer authority in a single entity?	DKARGO-016
	Are withdrawals delayed for a long period because the challenge protocol allows only partial concurrency and disputes are prolonged due to consecutive challenges?	N/A
	Is there a risk that the security level changes to a Permissionless model where anyone can propose a state because the allowlist is invalidated due to Validator inactivity?	DKARGO-006
	Is there a risk of loss due to incorrect state finalization because DAC members are few or lack diversity, leading to collusion to sign manipulated data or refusal to sign normal data?	DKARGO-003
	Is withdrawal impossible because nodes cannot secure data as DA data exists only off-chain and	DKARGO-003

	the DAC refuses to provide data or a failure occurs?	
	Does DA data transmission/reception stop when a Relayer failure occurs, and is recovery impossible due to the absence of an automatic replacement mechanism?	DKARGO-003
	Does batch submission stop or the verification path become paralyzed due to Keyset change failure?	DKARGO-015
	Is there a concern that user UX will degrade because a centralized Sequencer can delay or censor inclusion and Force Inclusion delay may be prolonged?	DKARGO-004 DKARGO-013
	Is there a possibility of user damage because the operator can extract MEV by observing transaction ordering?	DKARGO-004
	Do users lose the opportunity to exit because malicious upgrades are reflected immediately due to the absence of upgrade delay?	DKARGO-001
	Are bridge operations and fee calculations distorted because custom gas token requirements or price/template configurations are inappropriate?	DKARGO-008
	Do bridge compatibility and user display break due to non-standard token standards, metadata, and decimals?	DKARGO-009

	Does L2 token metadata parsing failure occur due to L1 metadata non-standardization?	DKARGO-018
	Are deposits/withdrawals routed incorrectly due to L1/L2 gateway router mapping discrepancies?	DKARGO-010
	Can external bridge adapters access the canonical bridge, exposing it to vulnerabilities upon security stack changes, thereby creating a risk of fund leakage?	N/A
	Can fast confirm, Validator, and bridge operations be manipulated if key theft or abuse occurs because key roles are concentrated in a single EOA?	DKARGO-001
	Does fund loss or service interruption occur in L3 because L3 relies on L2 security and L2 security stack issues propagate?	N/A
	Is the chain halted for a long time and are deposits, withdrawals, and trading impossible because configuration errors are reflected during operation?	N/A
	Are messaging/withdrawal paths interrupted because routing addresses (Inbox, Outbox, Sequencer) are set incorrectly?	DKARGO-017
	Does node verification fail due to rollup parameter (chainId/wasmModuleRoot/genesis) discrepancy?	DKARGO-011
Deployment Threat Based	Does a threat arise because the number of Validators and Fast Confirmers is limited during	DKARGO-006 DKARGO-016

	deployment, making state verification dependent on a single entity?	
	Does a threat arise because malicious or mistaken upgrades are reflected immediately as no upgrade delay is set?	DKARGO-001
	Does Validator participation decrease because the stakeToken/baseStake setting is inappropriate?	DKARGO-007
	Does a threat arise because the number of DAC members is small and the threshold is low, creating a structure where a small group can manipulate data availability?	DKARGO-003
	Does a threat arise because the DA bridge is stopped for a long time during a failure as the relayer replacement/addition function is not included in the design?	DKARGO-003
	Does exposure to censorship increase and user delay increase because delay and Force Inclusion parameters are set excessively?	DKARGO-005
	Are funds leaked due to adapter vulnerabilities or security stack changes because external bridge adapter authority is granted excessively?	N/A
	Do bridge initialization and operation fail because gateway/router paths or fee parameters are misconfigured?	DKARGO-010
	Are deposits/withdrawals stuck because bridge routing mapping is polluted due to deployment mapping override?	DKARGO-002

	Does initialization preemption or incorrect binding occur because deployment and initialization are separated?	DKARGO-012
	Is the chain halted or are key functions paralyzed because unaudited configuration changes are reflected?	N/A
	Does the deployment transaction fail or get delayed due to a retryable fee parameter calculation error?	DKARGO-014
	Does deployment revert or addresses mismatch due to template initializer compatibility issues or constructor bytecode changes?	DKARGO-011

3.2.2. Real-world Operation (On-chain) Based Threat Modeling

Classification	Content	ID Code
Operation Threat Based	Do malicious state root finalization and fund loss occur because challenges become impossible if block production and state updates stop and the DAC refuses to provide data?	DKARGO-003
	Is an incorrect state root finalized because verification is not performed due to collusion or absence of external independent Challengers, as they are not sufficiently distributed?	DKARGO-006
	Do withdrawals occur without a challenge period because malicious state roots are quickly finalized due to the Fast Confirmer being concentrated in a single key?	DKARGO-016
	Does incorrect state finalization occur because challenge initiation is delayed due to Validator/Challenger absence or inactivity?	DKARGO-006
	Are withdrawals repeatedly delayed because the challenge period is long and the attacker induces delay with consecutive challenges?	N/A
	Is a wrong DACert approved if a small group manipulates data signatures due to a low DAC threshold?	DKARGO-003
	Does state verification and withdrawal become impossible because DA data is not posted on-chain and the DAC does not provide data?	DKARGO-003

	Is recovery impossible because the relayer is authorized, causing the DA bridge to stop during a failure, and recovery relies on manual intervention?	DKARGO-003
	Do users lose the opportunity to exit upon malicious logic deployment because the structure reflects upgrades immediately?	DKARGO-001
	Does deployment revert or addresses mismatch due to template initializer compatibility issues or constructor bytecode changes?	DKARGO-011
	Is there a risk of fund theft because unverified source contracts are operated, potentially containing malicious logic?	N/A
	Does the trust premise change due to permissionless transition as the proposer becomes inactive for a long time?	DKARGO-006
	Is there a risk of fund leakage or bridge suspension because the routing contract becomes upgradeable by a single EOA and routing can be changed to a malicious path?	DKARGO-001 DKARGO-017
	Is immediate withdrawal impossible because external protocol dependency strategies are configured incorrectly, causing liquidity exhaustion or value decline?	N/A
	Does the threat of fund loss accumulate in the long term because the operating entity stops maintenance and response to new vulnerabilities is delayed?	N/A

	Does deployment/renewal fail or get delayed due to a retryable fee parameter calculation error?	DKARGO-014
	Is malicious upgrade possible upon key theft because UpgradeExecutor/ProxyAdmin authority is concentrated in a single path?	DKARGO-001
Deployment Threat Based	Does a threat arise because Validators, Challengers, and Fast Confirmers are concentrated in a few, making state verification dependent on a single entity?	DKARGO-006 DKARGO-016
	Does a threat arise because trust is concentrated in a few due to a low DAC threshold?	DKARGO-003
	Do users lose the opportunity to exit because upgrades are reflected immediately without delay?	DKARGO-001
	Does proposer drop or long-term suspension occur because the proposer inactivity criteria are set inappropriately?	DKARGO-006
	Is Force Inclusion delay excessive because delay and Force Inclusion parameters are set inappropriately?	DKARGO-005
	Is there a possibility of including and concealing malicious logic because unverified source contracts are deployed?	N/A
	Is deployment stopped due to rollup parameter discrepancy or template initialization failure?	DKARGO-011

	Is there a risk of fund loss upon key theft or abuse because key asset path contracts are controlled by a single administrator?	DKARGO-001
	Is the DA bridge halted for a long time because there is no relayer replacement path?	DKARGO-003

4. Findings

4.1. Findings Summary

As a result of the inspection, a total of 18 findings were derived. This report describes mitigation measures for deployment/operation and configuration threats and covers on-chain observation and assumption scenario-based threats together.

ID Code	Title	Type
DKARGO-001	Admin/Upgrade/FastConfirmer Authority Concentration and Abuse	Access Control
DKARGO-002	Abuse of Deployment mapping override	Access Control
DKARGO-003	DAS Keyset and DAC Operation Failure	Data Availability / DAC
DKARGO-004	Sequencer/BatchPoster Centralization and MEV Extraction	Sequencer / Delayed Inbox
DKARGO-005	Sequencer Inbox Parameter (Delay Buffer) Setting Insufficiency	Sequencer / Delayed Inbox
DKARGO-006	Validator Allowlist and Challenger Distribution Insufficiency	Validator / Assertion/Challenge
DKARGO-007	Staking Parameter Setting Discrepancy (stakeToken/baseStake)	Validator / Assertion/Challenge
DKARGO-008	Custom Gas Token Requirements, Pricing, and Template Approval Vulnerable Configuration	Access Control
DKARGO-009	Token Standard, Metadata, and Decimal Compatibility Vulnerable Configuration	Token Standard

DKARGO-010	Gateway/Router Path and Fee Operation Threats	Gateway / Router / Fee Operations
DKARGO-011	Rollup/Template Initialization Consistency Threats	Gateway / Router / Fee Operations
DKARGO-012	Insufficient Atomicity Assurance in Deployment and Initialization Process	Deployment / Initialization
DKARGO-013	Sequencer Delay/Censorship and Force Inclusion	Sequencer / Delayed Inbox
DKARGO-014	Retryable fee Parameter Calculation Error	Retryable / L1-to-L2 Messaging
DKARGO-015	AnyTrust/DAC Keyset Change Failure	Data Availability / DAC
DKARGO-016	Abuse of Fast Confirm Key	Access Control
DKARGO-017	Routing Address Misconfiguration (Inbox, Outbox, Sequencer)	Routing / Bridge Operations
DKARGO-018	L2 Token Metadata Parsing Failure	Token Standard

Classification by Type

Type	Occurring Threat	Count
Access Control	Threat of abuse of upgrade/state finalization due to concentration of key authorities in a single entity or incorrect authority paths.	4
Sequencer / Delayed Inbox	Threat of expanded censorship window, delay, and MEV/centralization due to batch submission and delay buffer misconfiguration.	3
Validator / Assertion/Challenge	Threat of weakened verification power due to lack of Validator/Challenger distribution and staking setting errors.	2
Data Availability / DAC	Threat of data availability loss and chain halt/censorship upon DAC configuration, Keyset change failure, or operation suspension.	2
Gateway / Router / Fee Operations	Threat of bridge path failure and operational failure due to routing, fee, and template setting errors.	2
Deployment / Initialization	Threat of initial setting preemption/error binding due to non-atomicity of deployment and initialization.	1
Retryable / L1-to-L2 Messaging	Threat of retransmission failure and deployment/operation delay due to fee calculation errors.	1
Routing / Bridge Operations	Threat of messaging/withdrawal path interruption due to address misconfiguration.	1

Token Standard	Threat of reduced UI/bridge compatibility and operational confusion due to non-standard token metadata/decimals.	2
Total		18

※ Detailed threat occurrence points can be found in the Target of individual findings.

4.2. Findings - On-chain Based Scenario Review

4.2.1. DKARGO-001: Admin/Upgrade/FastConfirmer Authority Concentration and Abuse

ID Code	DKARGO-001	Type	Access Control
Target	- ProxyAdmin.upgrade() - ProxyAdmin.upgradeAndCall() - UpgradeExecutor.execute() - RollupAdminLogic.setAnyTrustFastConfirmer() - RollupAdminLogic.setValidator() - RollupUserLogic.fastConfirmAssertion()		
Threat	- If the EXECUTOR_ROLE EOA of the UpgradeExecutor is compromised, the ProxyAdmin can be upgraded, immediately replacing the implementation of RollupProxy, Bridge, Inbox, and Outbox via ProxyAdmin.upgrade(). - If the anyTrustFastConfirmer key is compromised, a state root is finalized without a challenge period by calling RollupUserLogic.fastConfirmAssertion(). - If there is no upgrade delay mechanism, malicious upgrades are reflected immediately, exposing users to the risk of asset loss without an exit opportunity to withdraw assets.		
Description	- In L2, rollup.owner and proxyAdmin.owner are the same address (0xcc5bc4f409a9f50782eb39ea27bcd33740d28050). - UpgradeExecutor is AccessControl based, and EXECUTOR_ROLE is granted to an EOA (0x88b70778207c55132603602e6fe7d16c02573a63). - rollup.anyTrustFastConfirmer = 0xe5d803178c6fb5d86e88d7a41a47c26f15951eb5, and this		

address is a Gnosis Safe 1-of-1 (1 owner, threshold=1) configuration.

- The application of a Timelock for the **ProxyAdmin** path could not be confirmed on-chain.

```
# rollup owner / anyTrustFastConfirmer
cast call --rpc-url $ARB_RPC \
0x11e3D0e9604a0AD8a8B32068B95e83d7C63b3af7
"owner() (address) "
cast call --rpc-url $ARB_RPC \
0x11e3D0e9604a0AD8a8B32068B95e83d7C63b3af7
"anyTrustFastConfirmer() (address) "
# ProxyAdmin owner
cast call --rpc-url $ARB_RPC \
0x2582E771BC50f6273b17349780d3f8eB98A2b143
"owner() (address) "
# UpgradeExecutor roles (ADMIN_ROLE / EXECUTOR_ROLE)
cast keccak "ADMIN_ROLE"
cast keccak "EXECUTOR_ROLE"
cast call --rpc-url $ARB_RPC \
0xcC5Bc4F409a9F50782eb39Ea27bcd33740d28050
"hasRole(bytes32,address) (bool) " \
0xa49807205ce4d355092ef5a8a18f56e8913cf4a201fbe287825b095
693c21775 \
0xcC5Bc4F409a9F50782eb39Ea27bcd33740d28050
cast call --rpc-url $ARB_RPC \
0xcC5Bc4F409a9F50782eb39Ea27bcd33740d28050
"hasRole(bytes32,address) (bool) " \
0xd8aa0f3194971a2a116679f7c2090f6939c8d4e01a2a8d7e41d55e5
351469e63 \
0x88b70778207c55132603602e6fe7d16c02573a63
# Check for code existence at EXECUTOR_ROLE EOA
cast codesize --rpc-url $ARB_RPC \
0x88b70778207c55132603602e6fe7d16c02573a63
# anyTrustFastConfirmer (Gnosis Safe) member/threshold
cast call --rpc-url $ARB_RPC \
0xe5d803178c6fb5d86e88d7a41a47c26f15951eb5
"getOwners() (address[])"
cast call --rpc-url $ARB_RPC \
0xe5d803178c6fb5d86e88d7a41a47c26f15951eb5
"getThreshold() (uint256) "
```

	Admin/Upgrade/FastConfirmer On-chain Analysis Command:
Recommendation	There is a risk of Admin, Upgrade, and FastConfirmer authority concentration and abuse. The following measures should be applied to resolve the issue: 1. Deployment/Configuration Recommendation: • Maintain UpgradeExecutor as the Owner of ProxyAdmin, but convert the EXECUTOR_ROLE of UpgradeExecutor to a Multisig of 2-of-3 or higher. • Compose the Fast Withdrawals Committee (Multisig) centered on DAC operators, but clearly distinguish and define the roles of Fast Confirmer (Multisig address) and individual Validators (signer addresses) in the operation document. • Implement a Timelock-based upgrade process. 2. Operation Recommendation: • Upgrades and fast confirmation execution must undergo prior review and a multi-party approval process, and events such as Upgraded, OwnershipTransferred, and AnyTrustFastConfirmerSet should be monitored in real-time.
Reference	fast-withdrawals - docs.arbitrum.io

4.2.2. DKARGO-002: Abuse of Deployment mapping override

ID Code	DKARGO-002	Type	Access Control
Target	L1AtomicTokenBridgeCreator.setDeployment()		
Threat	- If the rollup owner calls <code>setDeployment()</code> incorrectly, the <code>inboxToL1Deployment</code> and <code>inboxToL2Deployment</code> mappings become polluted. - The router and gateway connect to incorrect addresses, causing deposits/withdrawals to become stuck.		
Description	In the <code>RollupCreated</code> event, the <code>upgradeExecutor</code> is set as the rollup owner, and this address holds the authority to call <code>setDeployment()</code>.		
Recommendation	There is a risk of abuse of deployment mapping override. The following measures should be applied to resolve the issue: 1. Deployment/Configuration Recommendation: - Reflect the Recommendation of DKARGO-001. - When calling <code>setDeployment()</code>, cross-verify the code existence of the input address and the expected deployment log. 2. Operation Recommendation: - Monitor the <code>OrbitTokenBridgeDeploymentSet</code> event and immediately verify against deployment logs and approval records when a change occurs.		
Reference	N/A		

4.2.3. DKARGO-003: DAS Keyset and DAC Operation Failure

ID Code	DKARGO-003	Type	Data Availability / DAC
Target	- SequencerInbox.setValidKeyset() - SequencerInbox.invalidateKeysetHash() - SequencerInbox.formCallDataHash() - SequencerInbox.addSequencerL2BatchFromOrigin()		
Threat	- Registering an invalid Keyset or invalidating a normal Keyset. <code>formCallDataHash()</code> reverts with <code>NoSuchKeyset</code>, stopping batch submission. - If DAC members are few, consensus diversity and liveness are low. A $k = 1$ structure allows the creation of a valid DACert with only a single signature, maximizing the risk of manipulation upon single key theft. - If there is no DA relayer replacement path, recovery is delayed in case of failure, causing a long-term chain halt.		
Description	Decoding the <code>SetValidKeyset</code> event reveals that the DAC Keyset is set to DAC members $(N) = 2$ and signature threshold $(\text{Threshold} / k) = 1$. With only 2 members, redundancy is lacking, leading to a high possibility of reduced Liveness. The signature threshold $(k) = 1$ configuration allows data manipulation with just a single key theft, presenting a risk from a Safety perspective.		

	<pre> # SequencerInbox address SEQ_INBOX=0x48781bAec9B5f9eBCf6fd96134f24231c6987Aa0 # 1. Check SetValidKeyset event topic cast keccak "SetValidKeyset(bytes32,bytes)" # 2. Check keysetHash validity (use hash extracted from event) cast call --rpc-url \$ARB_RPC \$SEQ_INBOX \ "isValidKeysetHash(bytes32) (bool)" \ 0x4800aa926dac8fc10a7e9263f87740eb669e242d6c55ca3404482da d54acf715 # 3. Check Keyset creation block cast call --rpc-url \$ARB_RPC \$SEQ_INBOX \ "getKeysetCreationBlock(bytes32) (uint256)" \ 0x4800aa926dac8fc10a7e9263f87740eb669e242d6c55ca3404482da d54acf715 # 4. Check reader4844 cast call --rpc-url \$ARB_RPC \$SEQ_INBOX \ "reader4844() (address)" # 5. Collect SetValidKeyset logs (Deployment range) cast logs --rpc-url \$ARB_RPC \ --address \$SEQ_INBOX \ --from-block 367605745 \ --to-block 367615744 \ "SetValidKeyset(bytes32,bytes)" </pre> DAS Keyset and DAC Operation On-chain Analysis Command:
Recommendation	There is a risk of DAS Keyset and DAC operation failure. The following measures should be applied to resolve the issue: 1. Deployment/Configuration Recommendation: Limit Keyset changes to multi-party approval and gradual rollover. Expand DAC members (N) to 3 or more and set the Threshold to 2/3 or more of the number of members (e.g., 2-of-3).

	• Configure automatic Calldata Fallback to operate if DACert creation fails, and design Relayer redundancy for stable data propagation. 2. Operation Recommendation: • Monitor <code>SetValidKeyset()</code>, <code>InvalidateKeyset</code> events, and DAC API availability in real-time. • Set it to automatically switch to rollup mode, which immediately posts full data to L1 Calldata if the DAC cannot provide sufficient signatures or an off-chain failure occurs. • Additionally, establish and check the Relayer switching process in advance for rapid recovery in case of failure.
Reference	anytrust-protocol - docs.arbitrum.io

4.2.4. DKARGO-004: Sequencer/BatchPoster Centralization and MEV Extraction

ID Code	DKARGO-004	Type	Sequencer / Delayed Inbox
Target	- SequencerInbox.setIsBatchPoster() - SequencerInbox.setBatchPosterManager() - SequencerInbox.addSequencerL2Batch()		
Threat	- If the Batch Poster is concentrated in a single key, calling <code>addSequencerL2Batch()</code> stops during failure or censorship. - If a centralized Sequencer monopolizes transaction ordering, there is a risk of exposure to Front-running and Sandwich Attacks for MEV extraction. - Force Inclusion is only possible after a delay, increasing user-perceived delay. - Including Delayed Inbox messages before Parent Chain Finalization increases Child Chain Reorg risk upon Parent Chain Reorg.		
Description	<code>batchPosterManager = address(0)</code>, and the Batch Poster confirmed by the <code>BatchPosterSet</code> event is set to a single address (<code>0x20e53116ac85ffd23cf102e777f409f58a73fad6</code>), presenting a centralization risk.		

	<pre> # 1. Check batchPosterManager cast call --rpc-url \$ARB_RPC \$SEQ_INBOX \ "batchPosterManager() (address)" # 2. Check maxTimeVariation (includes delaySeconds) cast call --rpc-url \$ARB_RPC \$SEQ_INBOX \ "maxTimeVariation() (uint256,uint256,uint256,uint256)" # 3. Check delay buffer cast call --rpc-url \$ARB_RPC \$SEQ_INBOX \ "buffer() (uint64,uint64,uint64,uint64,uint64,uint64)" # 4. Check isDelayBufferable cast call --rpc-url \$ARB_RPC \$SEQ_INBOX \ "isDelayBufferable() (bool)" # 5. BatchPosterSet event topic cast keccak "BatchPosterSet(address,bool)" # 6. BatchPosterSet logs cast logs --rpc-url \$ARB_RPC \ --address \$SEQ_INBOX \ --from-block 367605745 \ --to-block 367615744 \ "BatchPosterSet(address,bool)" # 7. Check Batch Poster status (Address confirmed from event) cast call --rpc-url \$ARB_RPC \$SEQ_INBOX \ "isBatchPoster(address) (bool)" \ 0x20e53116ac85ffd23cf102e777f409f58a73fad6 # 8. Check Batch Poster code existence (Check if EOA) cast codesize --rpc-url \$ARB_RPC \ 0x20e53116ac85ffd23cf102e777f409f58a73fad6 </pre> Sequencer/BatchPoster On-chain Analysis Command:
Recommendation	There is a risk of Sequencer/BatchPoster centralization and MEV extraction. The following measures should be applied to resolve the issue: 1. Deployment/Configuration Recommendation: Register at least 2 Batch Posters, and set the batchPosterManager, which has Poster addition and

	replacement authority, to a Multisig rather than a single key for management. 2. Operation Recommendation: • Monitor Delayed Inbox processing delay and Force Inclusion delay, and replace the Batch Poster or readjust parameters if delay increases. • Since users expect the Sequencer to collect transactions normally and include them in a fair order, the operation side should constantly monitor for signs of transaction delay or arbitrary reordering/censorship due to Sequencer/Batch Poster failure, malfunction, key leakage, or loss of control by external attack.
Reference	Sequencer - docs.arbitrum.io run-Sequencer-node - docs.arbitrum.io

4.2.5. DKARGO-005: Sequencer Inbox Parameter (Delay Buffer) Setting Insufficiency

ID Code	DKARGO-005	Type	Sequencer / Delayed Inbox
Target	- SequencerInbox.setMaxTimeVariation() - SequencerInbox.forceInclusion() - SequencerInbox.delayBufferableBlocks()		
Threat	<code>delayBlocks</code> or buffer values are set excessively. - The point at which <code>forceInclusion()</code> becomes possible is delayed, risking delayed Liveness recovery during failure (The emergency delay reduction function via Censorship Timeout does not work). - Users wait for a long time after submitting a message on L1.		
Description	<code>delaySeconds = 345600</code> means the <code>MaxTimeVariation</code> time boundary is set to 4 days, classified as an operational risk in terms of Batch Post and Reorg safety boundaries. <code>bufferBlocks</code> is configured with an excessively high value (2^{32}), so the Delay Buffer (variable delay) function that shortens the Force Inclusion waiting time is effectively disabled. Consequently, users must always wait the maximum waiting time (<code>delayBlocks</code>) in case of Sequencer censorship, meaning censorship resistance is not optimized. (<code>delayBlocks(28800)</code> is based on L1 blocks, which corresponds to <code>delaySeconds(345600)</code>.) <code>maxTimeVariation = [28800, 300, 345600, 3600]</code>, <code>buffer = [4294967296, 4294967296, 4294967296, 23124209, 500, 23124209]</code>, <code>isDelayBufferable = true</code>.		

	<pre> SEQ_INBOX=0x48781bAec9B5f9eBCf6fd96134f24231c6987Aa0 # 1. maxTimeVariation (delayBlocks, futureBlocks, delaySeconds, futureSeconds) cast call --rpc-url \$ARB_RPC \$SEQ_INBOX \ "maxTimeVariation() (uint256,uint256,uint256,uint256) " # 2. Delay buffer status (bufferBlocks, max, threshold, prevBlockNumber, replenishRateInBasis,prevSequencedBlockNumber) cast call --rpc-url \$ARB_RPC \$SEQ_INBOX \ "buffer() (uint64,uint64,uint64,uint64,uint64,uint64) " # 3. isDelayBufferable cast call --rpc-url \$ARB_RPC \$SEQ_INBOX \ "isDelayBufferable() (bool) " # 4. MaxTimeVariationSet event topic cast keccak "MaxTimeVariationSet (uint64,uint64,uint64,uint64) " # 5. MaxTimeVariationSet logs cast logs --rpc-url \$ARB_RPC \ --address \$SEQ_INBOX \ --from-block 367605745 \ --to-block 367615744 \ "MaxTimeVariationSet (uint64,uint64,uint64,uint64) " # 6. BufferConfigSet event topic cast keccak "BufferConfigSet((uint64,uint64,uint64)) " # 7. BufferConfigSet logs cast logs --rpc-url \$ARB_RPC \ --address \$SEQ_INBOX \ --from-block 367605745 \ --to-block 367615744 \ "BufferConfigSet((uint64,uint64,uint64)) " </pre> Delay buffer On-chain Analysis Command:
Recommendation	There is a risk of Delay buffer misconfiguration. The following measures should be applied to resolve the issue: 1. Deployment/Configuration Recommendation: Set delay/buffer values according to service recovery goals and limit changes to an approval process.

	- Set Delayed Inbox messages to <code>--node.delayed-Sequencer.require-full-finality=true</code> so they are included only after full finality. If <code>pre-finalization</code> is exceptionally permitted, document and manage the allowable <code>L1 reorg depth(=finalize-distance)</code> and risk acceptance range. 2. Operation Recommendation: - Monitor the <code>MaxTimeVariationSet</code> event and verify immediately upon change.
Reference	Sequencer - docs.arbitrum.io

4.2.6. DKARGO-006: Validator Allowlist and Challenger Distribution Insufficiency

ID Code	DKARGO-006	Type	Validator / Assertion/Challenge
Target	- RollupUserLogic.removeWhitelistAfterValidatorAfk() - RollupAdminLogic.setValidatorWhitelistDisabled() - RollupUserLogic.stakeOnNewAssertion() - EdgeChallengeManager.createLayerZeroEdge()		
Threat	- Validator/Challenger members are few, and the number of external independent actors is low. - Validators do not sufficiently verify the state root or collude. - Incorrect state roots are finalized without challenge, causing withdrawal losses. - If Validators are inactive and <code>removeWhitelistAfterValidatorAfk()</code> is called, the Allowlist is lifted, enabling a permissionless proposer. In this case, the attack surface expands where a malicious third party can propose an incorrect state root.		
Description	<code>validatorWhitelistDisabled = false</code>, <code>confirmPeriodBlocks = 50400</code>, <code>challengeGracePeriodBlocks = 14400</code>, <code>validatorAfkBlocks = 201600</code>. The list of addresses included in the Validator Allowlist was not confirmed on-chain. <pre> ROLLUP=0x11e3D0e9604a0AD8a8B32068B95e83d7C63b3af7 # Validator Allowlist status cast call --rpc-url \$ARB_RPC \ \$ROLLUP "validatorWhitelistDisabled() (bool)" cast call --rpc-url \$ARB_RPC \$ROLLUP \ "confirmPeriodBlocks() (uint64)" cast call --rpc-url \$ARB_RPC \$ROLLUP \ "challengeGracePeriodBlocks() (uint64)" cast call --rpc-url \$ARB_RPC \$ROLLUP \ "validatorAfkBlocks() (uint64)" </pre>		

	Validator Allowlist and Challenger On-chain Analysis Command:
Recommendation	There is a risk of insufficient Validator Allowlist and Challenger distribution. The following measures should be applied to resolve the issue: 1. Deployment/Configuration Recommendation: • Distribute Validator/Challenger count to N or more based on independent organizations. • Calculate N (Validator count) considering the threat model and operating costs/incentives, and set inactivity criteria conservatively according to operational reality. 2. Operation Recommendation: • Monitor state root submission, finalization events, and challenge initiation, and operate a Validator inactivity warning system.
Reference	choose-permissioned-validators - docs.arbitrum.io

4.2.7. DKARGO-007: Staking Parameter Setting Discrepancy (stakeToken/baseStake)

ID Code	DKARGO-007	Type	Validator / Assertion / Challenge
Target	- RollupCreator.createRollup() - RollupAdminLogic.setBaseStake()		
Threat	- Risk that the stakeToken setting does not match the current chain's challenge protocol version (e.g., Legacy Rollup vs. BoLD) and parent chain environment, causing deposit loss or making challenges impossible. - Initial baseStake is set excessively. - Entry costs for new Validators increase, lowering participation rates.		
Description	Although stakeToken and baseStake are specified simultaneously in the deployment settings, currently only the baseStake value (1e18) is confirmed, and stakeToken (address and Decimals) information is not finalized. Since the type and market value of stakeToken are not finalized, it cannot be judged whether the staking amount is appropriate. It must be recalculated considering the token's value to determine if it is at a level that prevents malicious challenges or if it is excessive enough to hinder Validator participation.		
Recommendation	There is a risk of staking parameter setting discrepancy. The following measures should be applied to resolve the issue: 1. Deployment/Configuration Recommendation: Fix the stakeToken to an asset suitable for the chain mode and restrict the change process. Set baseStake conservatively considering token decimals and economic simulation.		

	2. Operation Recommendation: Periodically check Validator participation metrics before and after changing baseStake.
Reference	stake-and-validator-configurations-staketoken - docs.arbitrum.io stake-and-validator-configurations-basestake - docs.arbitrum.io

4.3. Findings - Unverified On-chain Items and Assumption Based Scenario Review

4.3.1. DKARGO-008: Custom Gas Token Requirements, Pricing, and Template Approval Vulnerable Configuration

ID Code	DKARGO-008	Type	Access Control
Target	• ERC20Inbox.depositERC20() • ERC20Bridge.enqueueDelayedMessage() • DecimalsConverterHelper.adjustDecimals() • ArbOwner.setL1PricePerUnit() • ArbOwner.setL1PricingRewardRate() • L1AtomicTokenBridgeCreator.setTemplates() • L1AtomicTokenBridgeCreator.createTokenBridge() • L2AtomicTokenBridgeFactory._deployWethGateway() • RollupCreator.createRollup()		
Threat	• Fee token non-standard behavior and gas token requirement non-compliance: ◦ Fee-on-transfer or rebasing tokens are set as gas tokens. ◦ L1 escrow balance and L2 issuance amount mismatch. ◦ Collateral shortage occurs upon withdrawal. • Parent Chain base fee synchronization omission risk: ◦ When using a Custom Gas Token, if <code>setL1PricePerUnit(0)</code> and <code>setL1PricingRewardRate(0)</code> settings are not applied, and the Custom Gas Token is more expensive than ETH, excessive fees may be charged to the user. • Fee token pricer price calculation error: ◦ If the fee token pricer returns a stale/manipulated price when using Rollup mode + Custom Gas Token, the Batch		

	Poster is under/overcompensated, and the user bears excessive fees. • Template/fee-token mode mismatch: ○ If the fee token status and template configuration do not match, <code>NativeTokenMismatch</code> or incorrect gateway initialization occurs. • Fee-token template omission: ○ If the template address is 0, the L2 proxy connects to <code>address(0)</code> logic. • RollupCreator/L1AtomicTokenBridgeCreator approval shortage: ○ When using a Custom Gas Token, <code>createRollup()</code> fails if allowance for <code>RollupCreator</code> is insufficient. ○ L2/L3 token bridge deployment retryable fails if allowance for <code>L1AtomicTokenBridgeCreator</code> is insufficient.
Recommendation	There is a threat of Custom Gas Token requirements, pricing, and template approval vulnerable configuration. The following measures are proposed to resolve the issue: 1. Deployment/Configuration Recommendation: • The gas token must be a standard ERC-20, <code>transfer()</code> and <code>approve()</code> must directly call the token contract, and it must not have rebasing or fee-on-transfer properties. Also, there should be no callback/hook, and it should not revert when <code>sender == recipient</code>. • It should not revert on transfer of 0 value. • Fix the gas token address in the deployment settings and prohibit the change process. • Set <code>setL1PricePerUnit(0)</code>, <code>setL1PricingRewardRate(0)</code> via ArbOwner precompile.

	• Verify the match between <code>isUsingFeeToken</code> and template address. • Automatically verify that the template address is not 0. • Pre-approve the necessary Custom Gas Token allowance for <code>RollupCreator</code> and <code>L1AtomicTokenBridgeCreator</code> before deployment. 2. Operation Recommendation: • Perform actual deposit/withdrawal E2E tests before and after changing the gas token. • Clarify the pricer source and operate verification of oracle/tracking methods and monitoring for anomalies. • Fix the template address via an approval process before deployment. • Perform tests focused on key functions after changing the template.
Reference	use-a-custom-gas-token-anytrust - docs.arbitrum.io use-a-custom-gas-token-rollup - docs.arbitrum.io deploying-token-bridge - docs.arbitrum.io

4.3.2. DKARGO-009: Token Standard, Metadata, and Decimal Compatibility Vulnerable Configuration

ID Code	DKARGO-009	Type	Token Standard
Target	- DecimalsConverterHelper.adjustDecimals() - StandardArbERC20.bridgelnit() - StandardArbERC20.name() - StandardArbERC20.symbol() - StandardArbERC20.decimals() - ICustomToken.isArbitrumEnabled()		
Threat	Decimal rounding dust: - Accumulated error due to Rounding Down during low-decimal token bridging causes user-perceived loss. StandardArbERC20 metadata getter revert: - If the L1 token implements name, symbol, and decimals non-standardly, the L2 getter reverts. L1 Custom token isArbitrumEnabled unimplemented: - If the L1 token does not satisfy the <code>isArbitrumEnabled()</code> (returns 0xb1) requirement, custom gateway registration/bridging path fails.		
Recommendation	There is a threat of Token standard, metadata, and decimal compatibility vulnerable configuration. The following measures are proposed to resolve the issue: 1. Deployment/Configuration Recommendation: - Define a minimum withdrawal unit policy and expose it on the UI. - Pre-verify the metadata implementation of the L1 token. - Verify that custom tokens comply with <code>ICustomToken</code> and that <code>isArbitrumEnabled()</code> returns the correct value.		

	2. Operation Recommendation: • Operate a separate minimum deposit/withdrawal policy for low-decimal tokens. • Provide handling values for revert errors on the frontend.
Reference	token-bridge-erc20 - docs.arbitrum.io

4.3.3. DKARGO-010: Gateway/Router Path and Fee Operation Threats

ID Code	DKARGO-010	Type	Gateway / Router / Fee Operations
Target	• L1GatewayRouter.setGateway() • L2GatewayRouter.setGateway() • L1GatewayRouter._setGateways() • L1OrbitGatewayRouter.setGateway() • L1OrbitERC20Gateway.outboundTransfer() • L1AtomicTokenBridgeCreator.setTemplates() • L2AtomicTokenBridgeFactory._deployWethGateway()		
Threat	• L1/L2 gateway router mapping mismatch: ◦ Retryable fails after L1 router mapping change. ◦ L1/L2 router mapping does not match. • Orbit fee-token gateway fee/approval failure: ◦ If <code>_feeAmount</code> is underestimated or approval is insufficient, retryable creation fails. • Token bridge template/WETH address misconfiguration: ◦ If WETH/template address is incorrect, gateway initialization fails.		
Recommendation	There are threats to Gateway/Router path and fee operation. The following measures are proposed to resolve the issue: 1. Deployment/Configuration Recommendation: • Check for correct L2 mapping when changing the router. • Proceed with pre-verification of approval/balance for <code>_feeAmount</code>. • Cross-verify WETH/template addresses by chain. 2. Operation Recommendation: • Operate retryable failure notification and retransmission processes.		

	• Check retryable execution logs after changing fee parameters. • Perform tests focused on WETH deposit/withdrawal key functions after deployment.
Reference	token-bridge-erc20 - docs.arbitrum.io deploying-token-bridge - docs.arbitrum.io canonical-factory-contracts - docs.arbitrum.io

4.3.4. DKARGO-011: Rollup/Template Initialization Consistency

Threats

ID Code	DKARGO-011	Type	Gateway / Router / Fee Operations
Target	- RollupCreator.createRollup() - RollupAdminLogic.initialize() - StandardArbERC20.bridgeInit() - L2GatewayToken._initialize() - L2AtomicTokenBridgeFactory._deployWethGateway() - CreationCodeHelper.getCreationCodeFor() - L2AtomicTokenBridgeFactory._deployCustomGateway()		
Threat	Rollup parameter mismatch: - Node verification fails due to mismatch in <code>chainId</code>, <code>wasmModuleRoot</code>, <code>genesis</code> parameters. StandardArbERC20 Non-atomic initialization: - If proxy deployment and <code>bridgeInit()</code> calls are separated, a third party can initialize first. aeWETH/UpgradeExecutor constructor bytecode mismatch: - CREATE2 deployment fails or the generated address does not match expectations due to init code hash changes resulting from runtime code/creation code changes. Template initializer compatibility issue: - Deployment reverts if the template logic does not allow dummy initialization.		
Recommendation	There is a threat of Rollup/Template initialization integration. The following measures are proposed to resolve the issue: 1. Deployment/Configuration Recommendation: Fix and verify deployment parameters as a single JSON.		

	• Perform proxy deployment and initialization in a single transaction using <code>L2AtomicTokenBridgeFactory</code>. • Pre-calculate deployment addresses (deterministic addresses) using CREATE2 and verify if they match the actual deployment. • Maintain dummy initialization interface during template upgrades. 2. Operation Recommendation: • Perform pre-tests before and after deployment. • Enforce <code>bridgelnit()</code> call in custom deployment scripts. • Perform deployment pre-tests before and after changing templates.
Reference	deploying-an-arbitrum-chain - docs.arbitrum.io canonical-factory-contracts - docs.arbitrum.io deploying-token-bridge - docs.arbitrum.io

4.4. Findings - Contract Scenario Based Review

4.4.1. DKARGO-012: Insufficient Atomicity Assurance in Deployment and Initialization Process

ID Code	DKARGO-012	Type	Deployment / Initialization
Target	- RollupProxy.initializeProxy() - StandardArbERC20.bridgeInit()		
Threat	- Setting admin/user logic and owner via <code>RollupProxy.initializeProxy()</code>. - Initializing <code>StandardArbERC20.bridgeInit()</code> after L2 token creation. - Since the initialization function is public, there is a possibility of Front-running by an attacker, which may disrupt normal initialization.		
Recommendation	To design for Atomicity assurance in deployment and initialization, the following measures are proposed: If deployment and initialization are not performed atomically, incorrect admin/gateway binding is possible. It is proposed to establish a single transaction or immediate initialization process at the deployment stage.		
Reference	canonical-factory-contracts - docs.arbitrum.io deploying-token-bridge - docs.arbitrum.io		

4.4.2. DKARGO-013: Sequencer Delay/Censorship and Force Inclusion

ID Code	DKARGO-013	Type	Sequencer / Delayed Inbox
Target	• ERC20Inbox.depositERC20() • ERC20Bridge.enqueueDelayedMessage() • SequencerInbox.addSequencerL2BatchFromOrigin() • SequencerInbox.addSequencerL2Batch() • SequencerInbox.addSequencerL2BatchFromOriginDelayProof() • SequencerInbox.addSequencerL2BatchDelayProof() • SequencerInbox.forceInclusion()		
Threat	• Force Inclusion should be utilized as an emergency path, but inappropriate Delay buffer settings can unintentionally affect the Force Inclusion window.		
Recommendation	For the safe design of Sequencer Delay/Censorship and Force Inclusion, the following measures are proposed: • Since Force Inclusion should only be operated as an emergency path, optimize Delay buffer and Threshold settings according to operational goals so that flexible responses are possible depending on the failure situation.		
Reference	Sequencer - docs.arbitrum.io		

4.4.3. DKARGO-014: Retryable fee Parameter Calculation Error

ID Code	DKARGO-014	Type	Retryable / L1-to-L2 Messaging
Target	- ERC20Inbox.createRetryableTicket() - ERC20Inbox.unsafeCreateRetryableTicket()		
Threat	- Attempting to create retryable via ERC20Inbox.createRetryableTicket(), ERC20Inbox.unsafeCreateRetryableTicket(). - Revert occurs if fee parameters are insufficient.		
Recommendation	For the safe design of Retryable fee, the following measures are proposed: - Since gas/fee calculation errors lead to retryable creation failure and increased user retry costs, establishing a standard gas estimator and operational criteria is proposed. - When using a Custom Gas Token, verifying that the ticket creation requirement of ERC20Inbox (<code>tokenAmount (18-decimals normalized value) >= maxSubmissionCost + l2CallValue + gasLimit * maxFeePerGas</code>) is met is proposed. Also, establishing a manual redeem guide and operational process for users to manually execute in case of auto-redeem failure is proposed.		
Reference	l1-to-l2-messaging - docs.arbitrum.io		

4.4.4. DKARGO-015: AnyTrust/DAC Keyset Change Failure

ID Code	DKARGO-015	Type	Data Availability / DAC
Target	- SequencerInbox.setValidKeyset() - SequencerInbox.invalidateKeysetHash() - SequencerInbox.addSequencerL2BatchFromOrigin() - SequencerInbox.addSequencerL2Batch() - SequencerInbox.addSequencerL2BatchDelayProof()		
Threat	- Managing Keyset via <code>SequencerInbox.setValidKeyset()</code>, <code>SequencerInbox.invalidateKeysetHash()</code>. - Applied to batch submission paths (<code>addSequencerL2Batch*</code>). - Keyset change directly affects batch submission.		
Recommendation	For the safe design of AnyTrust/DAC Keyset, the following measures are proposed: - Since Keyset change mistakes lead to batch submission suspension, a Keyset rotation process and pre-testing are proposed. - Since AnyTrust relies on DAC Keyset/threshold, defining the number of members and threshold in advance, and determining Keyset changes before deployment or performing pre-tests when changing during operation is proposed.		
Reference	anytrust-protocol - docs.arbitrum.io configure-dac - docs.arbitrum.io		

4.4.5. DKARGO-016: Abuse of Fast Confirm Key

ID Code	DKARGO-016	Type	Access Control
Target	- RollupAdminLogic.setAnyTrustFastConfirmer() - RollupUserLogic.fastConfirmAssertion() - RollupUserLogic.fastConfirmNewAssertion()		
Threat	- Setting authority via <code>RollupAdminLogic.setAnyTrustFastConfirmer()</code>. - Executing <code>RollupUserLogic.fastConfirmAssertion()</code>, <code>RollupUserLogic.fastConfirmNewAssertion()</code>.		
Recommendation	For the safe design of the Fast Confirm Key, the following measures are proposed: 1. Since early finalization abuse is possible upon Fast Confirmer key leakage/abuse, multisig operation and signing policies are proposed. 2. Operating Fast Confirmer authority as a Permissioned Committee (Multisig), recommending at least 3 Validators and at least 3 DAC members, and configuring it so that Validators registered in the Allowlist participate in the committee is proposed.		
Reference	fast-withdrawals - docs.arbitrum.io choose-fast-withdrawals - docs.arbitrum.io		

4.4.6. DKARGO-017: Routing Address Misconfiguration (Inbox, Outbox, Sequencer)

ID Code	DKARGO-017	Type	Routing / Bridge Operations
Target	- RollupAdminLogic.setOutbox() - RollupAdminLogic.setInbox() - RollupAdminLogic.setSequencerInbox() - ERC20Bridge.enqueueDelayedMessage() - ERC20Outbox.executeTransaction()		
Threat	- Changing routing via <code>RollupAdminLogic.setOutbox()</code>, <code>RollupAdminLogic.setInbox()</code>, <code>RollupAdminLogic.setSequencerInbox()</code>. - Affects <code>ERC20Bridge.enqueueDelayedMessage()</code> and <code>ERC20Outbox.executeTransaction()</code> paths. - Routing address change affects overall messaging/withdrawal.		
Recommendation	For the safe design of routing address setting (Inbox, Outbox, Sequencer), the following measures are proposed: Since incorrect address settings immediately stop deposit/withdrawal/messaging paths, establishing a verification process before and after address changes is proposed.		
Reference	deploying-an-arbitrum-chain - docs.arbitrum.io token-bridge-erc20 - docs.arbitrum.io		

4.4.7. DKARGO-018: L2 Token Metadata Parsing Failure

ID Code	DKARGO-018	Type	Token Standard
Target	StandardArbERC20.bridgeInit()		
Threat	Parsing metadata in StandardArbERC20.bridgeInit() and calling name(), symbol(), decimals().		
Recommendation	For the safe design of L2 token metadata parsing, the following measures are proposed: Since L2 token metadata becomes permanently unavailable if L1 metadata is unsupported/parsing fails, L1 metadata verification is proposed.		
Reference	token-bridge-erc20 - docs.arbitrum.io		

5. Discussion

5.1. Operational Maturity Risk Management

This section summarizes operational maturity focusing on components identifiable on-chain. In other words, it organizes normal configurations that can be confirmed, such as standard bridge mappings and pairings, together with operational assumptions requiring additional checks, such as multisig/authority distribution status.

The following items are those confirmed on-chain that meet Arbitrum Best Practices or match standard settings.

#	Observation Result
1	<code>SequencerInbox.reader4844 = 0x0</code> and does not fall under the <code>DataBlobsNotSupported()</code> condition on the Arbitrum host.
2	<code>L1 DKA decimals() = 18</code> , <code>L2 DKA decimals() = 18</code> , and <code>bridge.nativeToken</code> matches L2 DKA. <code>decimals=18</code> is confirmed, and compliance with custom gas token requirements depends on separate conditions (non-rebasing, non-fee-on-transfer, non-callback, etc.).
3	The <code>l1Address()</code> of L2 DKA matches the L1 DKA address, and the <code>calculateL2TokenAddress(L1 DKA)</code> result matches the actual L2 DKA address, so the L1<>L2 token mapping is normal.
4	L2<>L3 standard gateway pairings mutually match and <code>cloneableProxyHash</code> is identical. The L3 <code>beaconProxyFactory</code> is confirmed, and the standard <code>StandardArbERC20</code> deployment path is normal.

The following is a list classified based on the multisig/authority configuration status among dKargo on-chain assets.

Asset / Chain	Rollup / L2
Address	0x11e3D0...3b3af7

Classification	EIP-1967 proxy
Authority Configuration	owner=UpgradeExecutor, EXECUTOR_ROLE=EOA
Risk Verdict	Authority Concentration / Validator Distribution Unconfirmed

Asset / Chain	UpgradeExecutor / L2
Address	0xC5Bc4...d28050
Classification	EIP-1967 proxy
Authority Configuration	EXECUTOR_ROLE=EOA
Risk Verdict	Upgrade Authority Concentration

Asset / Chain	ProxyAdmin / L2
Address	0x2582E7...A2b143
Classification	Contract
Authority Configuration	owner=UpgradeExecutor
Risk Verdict	Absence of Upgrade Delay

Asset / Chain	anyTrustFastConfirmer / L2
Address	0xe5d803...951eb5
Classification	Gnosis Safe

Authority Configuration	1-of-1 Safe
Risk Verdict	Fast confirm single key risk

Asset / Chain	SequencerInbox / L2
Address	0x48781b...987Aa0
Classification	EIP-1967 proxy
Authority Configuration	admin=ProxyAdmin
Risk Verdict	Batch Poster: Single Address, DAC: Small number configuration

Asset / Chain	Bridge / L2
Address	0x42C4b4...98e162
Classification	EIP-1967 proxy
Authority Configuration	admin=ProxyAdmin
Risk Verdict	Dependence on fee token standardization

Asset / Chain	DKA / L1
Address	0x5dc60C...5efCE0
Classification	ERC20

Authority Configuration	N/A
Risk Verdict	No Risk (Basic specification confirmed)

Asset / Chain	DKA / L2
Address	0x1E2C41...e54F1C
Classification	ERC20(IArbToken)
Authority Configuration	N/A
Risk Verdict	No Risk (Mapping confirmed)

Asset / Chain	StandardGateway Pair / L2/L3
Address	L2 0x306485...c7A289, L3 0xE400CA...A1e246
Classification	Paired
Authority Configuration	admin=L2/L3 ProxyAdmin
Risk Verdict	No Risk (Pairing Normal)

Asset / Chain	ProxyAdmin / L3
Address	0x5CF9C1...7E6c5A
Classification	Contract

Authority Configuration	Multisig Status Unconfirmed
Risk Verdict	Authority Configuration Unconfirmed

Among these, excluding items with no risk based on on-chain observation results, 4 cases of Authority/Governance, 2 cases of Operation/Configuration, and 1 case of Unconfirmed/Unknown were identified. Accordingly, the mitigation measures suggested in the findings should be applied first, and unconfirmed items require additional verification.

Below is the list of findings related to these risks.

ID Code	Title
DKARGO-001	Admin/Upgrade/FastConfirmer Authority Concentration and Abuse

5.2. Gelato RaaS Operational Risk Management

This section summarizes the prerequisites and management items required for chain operation in the Gelato RaaS environment. Since Gelato acts as an operational model providing infrastructure and essential Web3 integration, responsibility sharing and failure response scopes must be clearly defined in advance.

Prerequisites are divided into custom gas token requirements and Verifier Node operation requirements. First, when using a custom gas token, the ERC-20 address must be deployed on the parent chain, and `transfer()` and `approve()` must be direct calls to the token contract. Also, conditions such as non-rebasing, non-fee-on-transfer, no callback/hook, and prohibition of revert when `sender == recipient` must be met, and the token address must be included in the deployment configuration, with the change process restricted by operational policy.

Next, Verifier Node operation assumes a NodeKey NFT, which must be minted with the chain's gas token and then delegated to the Operator via the `DelegateRegistry`. Setting the Gelato API Key and `L1_RPC_URL` is recommended, and server operation based on cloud run job scheduling is suggested as an example. The NodeKey, Referee, NodeRewards, and node clients included in the Verifier Node are provided as audited baselines, but customization requires verification at the same level.

Operational Considerations When Changing DA Selection

The content below concerns potential considerations applicable only when changing the DA layer from AnyTrust to an external DA or adding a Fallback configuration. Since changing DA selection alters data availability, recovery paths, and the scope of operational responsibility, prior criteria and transition processes must be clear. The following summarizes operational considerations by DA option.

1. If adopting Celestia (DAS/NMT based) as a DA solution, transaction data can be posted to the Celestia network instead of Parent Chain calldata. During downtime, a fallback configuration to another DA or Parent Chain calldata must be in place.

2. Avail provides DA based on DAS + erasure coding and does not require Validator set/tokenomics operation. The scope of DA availability responsibility must be clearly defined by operational policy.
3. EigenDA relies on Operators, Disperser, and Retrievers components, so availability monitoring and failure response processes for each configuration are required.

5.3. Operational Risk Management Based on Similar Project Incident Cases

The cases below show how similar operational risks can materialize based on risks that actually occurred during operation.

1. Re.al chain Case: DAC Operation Risk

AnyTrust stores data in the DAC instead of L1 calldata to reduce DA costs and posts only the DACert, a proof of data storage, to L1. The trust assumption of this structure is that at least one DAC member is honest. Re.al's DAC was operated with a 2-person configuration and a 1/2 threshold (1-of-2), but the transition to Fallback (L1 data posting) was not smooth during the inactivity of DAC members, leading to a chain halt. Also, if the DAC fails to provide data during the forced withdrawal delay period of about 12 days, challenges become impossible, increasing the risk of fund theft via malicious upgrades. The absence of an exit window and the immediate upgrade structure amplify these risks. In other words, since the number of DAC members and operational continuity are key premises of AnyTrust availability, the minimum number of members and the entity responsible for operation must be clearly defined.

2. Degen Chain Case: RaaS Centralization and SPoF Risk

According to public reports, Conduit applied an unverified configuration change that increased the batch size to 10MB. During the rollback process, nodes failed to process data, and the database was corrupted, requiring resynchronization. As a result, a long-term outage of about 50-54 hours and a reorg of about 500,000 blocks occurred. In a RaaS operating environment, unverified configuration changes can lead to data posting delays and finality risks. Therefore, changes to batch and Force Inclusion parameters must be controlled through pre-testing and approval processes. In other words, batch size and the Force Inclusion window must be verified and managed together, and criteria for detecting and responding to posting delays must be clearly defined by operational policy.

Below is the list of findings related to the risks of these incident cases.

ID Code	Title
DKARGO-003	DAS Keyset and DAC Operation Failure
DKARGO-004	Sequencer/BatchPoster Centralization and MEV Extraction
DKARGO-005	Sequencer Inbox Parameter (Delay Buffer) Setting Insufficiency

6. Conclusion

As a result of the inspection, the design security and deployment/operation configuration of the dKargo L3 chain were reviewed, and operational risks in the Gelato RaaS environment were evaluated.

In the on-chain configuration, some standard mappings and pairings were confirmed to be normal, but risks such as concentration of upgrade and state finalization authority, DAC operational configuration, insufficient distribution of Sequencer/batch submission operations, and lack of control over operational setting value changes were identified.

The findings of this report focus on proposing measures to secure operational stability and long-term reliability. Therefore, it is recommended to prioritize the application of authority distribution (Multisig/Timelock), strengthening of DAC members and thresholds and monitoring systems, distribution of Sequencer/batch submission operations, and pre-deployment verification checklists and approval procedures.

Especially in the Gelato RaaS environment, the sharing of operational responsibilities and the scope of failure response must be clearly defined, and custom gas token and Verifier Node operation requirements must be documented to manage operational risks.

In the long term, operational security processes combining threat modeling and change approval procedures should be established at the design stage, and the impact of deployment/operation changes on chain stability should be continuously verified.

Through this, dKargo will be able to stably secure the availability, censorship resistance, and asset safety of the L3 chain. By internalizing security throughout the planning, development, and operation stages through these efforts, the dKargo service will be able to identify and respond to potential threats early and build a stable and highly reliable chain.



Theori, Inc. ("We") is acting solely for the client and is not responsible to any other party.

Deliverables are valid for and should be used solely in connection with the purpose for which they were prepared as set out in our engagement agreement. You should not refer to or use our name or advice for any other purpose.

The information (where appropriate) has not been verified. No representation or warranty is given as to accuracy, completeness or correctness of information in the Deliverables, any document, or any other information made available. Deliverables are for the internal use of the client and may not be used or relied upon by any person or entity other than the client. Deliverables are confidential and are not to be provided, without our authorization (preferably written), to entities or representatives of entities (including employees) that are not the client, including affiliates or representatives of affiliates of the client

©2026. For information, contact Theori, Inc.